MPPLAB · C-DAC PUNE

DATA ANALYSIS OF STREAMING TELECOM TRAFFIC DATA USING DATA EXTRACTION SERVER IN A MESSAGE PASSING ENVIRONMENT

MPI-Based Multi-DES Communication for Distributed Telecom Packet Processing

Prepared by **Pathi Jahnavi (2023UCP1595)** — *Team Lead* · *MNIT Jaipur*

Katta Sreeja Meharani (23251A6750) · *GNITS Hyderabad*

Project Guidance **MPPLAB (NSM Project) Experts**

Centre for Development of Advanced Computing (C-DAC), Pune — MPPLAB

May – July 2026

INTRODUCTION

Telecom Streams Have Outgrown Batch Processing

Data Explosion

5G and LTE networks generate continuous, high-velocity packet streams that defy traditional batch processing.

Real-Time Demands

Providers need sub-millisecond analytics for anomaly detection, billing, and network optimisation.

Scalability Necessity

Handling large-scale packet volumes demands distributed, parallel computing to avoid bottlenecks.

THE SOLUTION

The Hybrid Telecom Stream Processing Framework integrates distributed Data Extraction Servers (DES) with a parallelised MPI-based backend for reliable, high-throughput packet handling — from ingestion to storage, analytics, and monitoring.

THE CHALLENGE

Telecom Infrastructure Buckles Under Peak Traffic

Distributed Complexity

Synchronising traffic across multiple, concurrently active Data Extraction Servers.

Communication Latency

Conventional messaging protocols struggle to keep pace with high-speed packet ingestion.

Concurrency Overhead

Shared resources must be managed without compromising ordering or throughput.

Resource Constraints

Need for continuous runtime monitoring and non-blocking, reliable storage.

Monolithic architectures fail under bursty traffic — resulting in packet drops and degraded QoS.

STRATEGIC OBJECTIVES

What the Framework Set Out to Achieve

Communication Architecture

Develop an MPI-based framework for reliable, low-latency communication between distributed nodes.

Scalable Extraction

Support multiple, concurrently active DES clients without redesigning the server's core logic.

Fine-Grained Parallelism

Use POSIX Threads for concurrent, high-throughput packet processing within each node.

Data Reliability

Integrate Berkeley DB for persistent, high-speed storage of every processed packet.

SYSTEM DESIGN

Integrated System Architecture

Data Extraction Servers (DES) → MPI Communication → Processing Tier (POSIX Threads) → Berkeley DB Persistence → Graph Construction → Monitoring Dashboard

A decoupled producer–consumer pipeline: MPI decouples ingestion from processing, so a burst of incoming DES traffic never blocks the communication layer.

Modular

Independently Testable

Composable

UNDER THE HOOD

Core Technology Stack

C	Language
POSIX Threads	Parallelism
MPI (OpenMPI)	Distributed Comms
TCP Sockets	Early-Phase Networking
Berkeley DB	Storage
Mutex / CondVar	Synchronisation
Queue & Graph	Data Structures
GNU Make	Build System
Ubuntu Linux	Operating System
Git & GitHub	Version Control

DEVELOPMENT JOURNEY

Implementation Milestones

1

POSIX Threads

Worker-pool design for task parallelisation.

2

TCP Sockets

Socket-based connectivity between local nodes.

3

Socket + Thread Integration

Thread-safe queues combined with socket ingestion.

4

MPI Backbone

Migration to MPI for multi-node scalability.

5

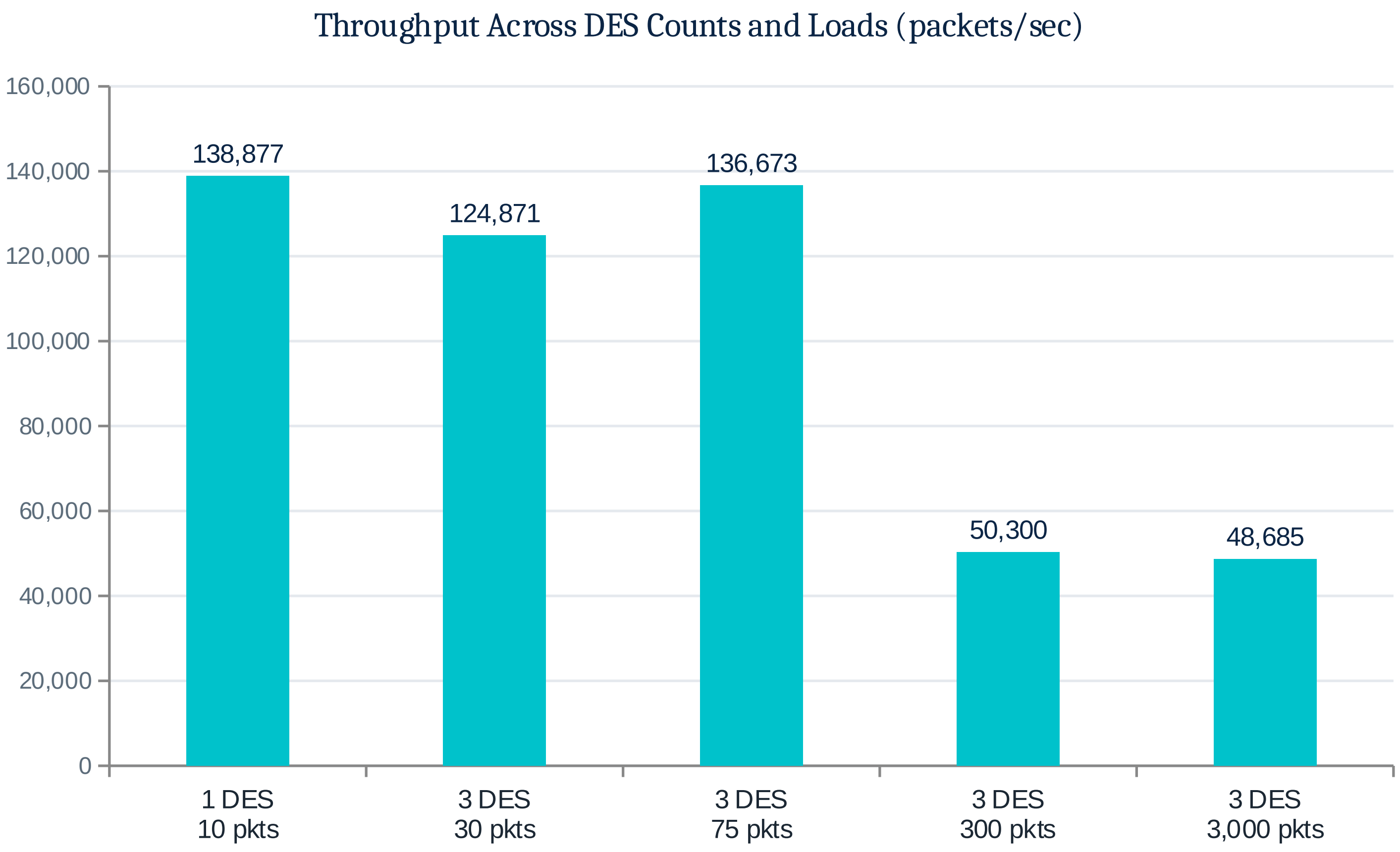
Hybrid Integration

Final integration with Berkeley DB and monitoring.

Each phase — independently designed, implemented, tested, and validated — before progressing to the next.

RESULTS

Performance & Throughput Under Load



KEY FINDINGS

- Stable 48.7K–50.3K pkts/sec sustained across a 10x jump in load (300 → 3,000 pkts)
- Under 4% throughput variance despite the tenfold increase in packet volume
- Zero packet loss, deadlocks, or race conditions at every scale tested

PROOF IT WORKS

Validation & Quality Assurance

Integrity

FIFO packet sequencing maintained throughout the entire stream pipeline, in every configuration tested.

Synchronisation

Extensive stress testing across five load levels — no deadlocks or race conditions observed.

Functional Coverage

Producer/consumer modules and MPI-DES handshake fully validated under synthetic traffic generators.

5 / 5

Test Configs Passed

0

Packets Lost

3,000

Peak Packets Validated

WHAT'S NEXT

Future Research & Scaling

Cloud Integration

Transition to a Kubernetes-managed, containerised deployment.

Predictive Analytics

AI/ML models for proactive traffic prediction and load balancing.

Fault Tolerance

Checkpointing mechanisms to withstand node failure at scale.

Visualization

GUI-based analytics dashboard for real-time network state.

SUMMARY

A Production-Ready Foundation

The Hybrid Telecom Stream Processing Framework delivers a high-performance, scalable, and reliable architecture for modern telecom packet analytics — decoupling extraction from processing via MPI and POSIX Threads to solve the latency and concurrency challenges of massive data streams.

48.7K+	pkts/sec sustained throughput
3×	concurrent DES clients validated
0	packets lost across all tests

Thank You

Questions & Discussion

Pathi Jahnavi (2023UCP1595) — *Team Lead*

Katta Sreeja Meharani (23251A6750)

Guided by MPPLAB (NSM Project) Experts — C-DAC, Pune